Test-Centrism

Yohei Watanabe

Abstract

This paper generalizes the idea of Test-Driven Development (TDD) in software engineering and proposes a framework for understanding concepts, propositions, and theories through tests.

In TDD, program specifications are described by tests, and correctness is judged based on how the program behaves with respect to those tests. This idea is not limited to software development but can be understood as a more general framework for cognition.

Similar ideas exist in philosophy. In pragmatism, Charles S. Peirce argued that the meaning of an idea should be clarified in terms of its practical consequences. In the philosophy of science, Karl Popper proposed falsifiability as a criterion for scientific theories, maintaining that a theory must be capable of being proven false by observation.

This paper unifies these perspectives and calls the stance of understanding objects through tests **Test-centrism**.

In Test-centrism, a test is treated as a finite ordered sequence, and each test consists of:

- an input
- an observed value (if available) or an undefined value (open)

Within this framework, we distinguish two relations concerning tests:

- **Discriminability**: the behavior of an object is defined for the input of a test
- **Verifiability**: the behavior of an object can be compared with an observed result

These are defined independently of the type of object (concept, proposition, or theory). Therefore, this paper treats both discrimination and verification uniformly for all objects.

Furthermore, the meaning of an object is defined as its behavior restricted to a test sequence. This framework also allows concepts such as understanding and refactoring to be formalized within the same structure.

Test-centrism is formalized categorically. We construct a category whose objects are test sequences, and the behavior of an object is represented as a contravariant functor:

$$F_T : \mathcal{X}^{op} \to \mathbf{Set}$$

In this setting, the meaning of an object is given as a presheaf, and the Test-centrism category

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}$$

forms a presheaf topos.

The internal logic of this topos is intuitionistic logic, which can further be interpreted via Kripke semantics. In this interpretation, worlds are understood as test sequences and truth as a property stable under test extensions.

Within this framework, we show that:

- concepts (practical meaning)
- programs (computational objects)
- scientific theories (predictive systems)

can all be uniformly described as **responses to tests**.

Contents

1 Introduction 6

2 TDD 6
- 2.1 Test-Driven Development . . . 6
- 2.2 Refactoring . . . 7
- 2.3 What is a Test? . . . 8
- 2.4 Example of TDD . . . 8
  - 2.4.1 STEP 1 — The first test . . . 8
  - 2.4.2 STEP 2 — Adding a test . . . 9
  - 2.4.3 STEP 3 — A failing test . . . 9
  - 2.4.4 STEP 4 — The Problem of Input Order . . . 10
  - 2.4.5 STEP 5 — When Division Does Not Complete in One Step . . . 10
  - 2.4.6 STEP 6 — Coprime case . . . 11

3 Pragmatic Maxim 13
- 3.1 Encoding the Pragmatic Maxim in TypeScript . . . 13
  - 3.1.1 Why “hard” should be an interface, not a class . . . 17

4 Falsifiability 18

5 Toward a Formalization of Test-Centrism: Some Examples 19
- 5.1 Greatest Common Divisor . . . 19
  - 5.1.1 Behavioral Function . . . 19
  - 5.1.2 Tests . . . 19
  - 5.1.3 Test Sequences . . . 20
  - 5.1.4 Test Results . . . 21
- 5.2 Example of Pragmatism . . . 22
- 5.3 Example of a Scientific Theory . . . 23

6 Test-Centrism 24
- 6.1 Basic Idea . . . 24
- 6.2 The Role of Tests . . . 24
  - 6.2.1 Tests for Construction . . . 24
  - 6.2.2 Tests for Understanding . . . 24

7 Formalization 24
- 7.1 Axiom 1 (Objects) . . . 25
- 7.2 Axiom 2 (Behavioral Function) . . . 25
- 7.3 Axiom 3 (Tests) . . . 25

7.4 Axiom 4 (Test Sequence) . . . 25
7.5 Axiom 5 (Meaning) . . . 26
7.6 Axiom 6 (Verifiability) . . . 26
7.7 Axiom 7 (Discriminative Equivalence) . . . 26
7.8 Axiom 8 (Verification Equivalence) . . . 26
7.9 Axiom 9 (Understanding) . . . 27
7.10 Axiom 10 (Complexity) . . . 27
7.11 Axiom 11 (Refactoring) . . . 27

8 Examples 27
8.1 Greatest Common Divisor: An Example of Verifiable Objects . . . 27
8.2 The Pragmatic Maxim: Hardness . . . 29
8.3 Scientific Theory: Falsifiability . . . 30

9 Categorical Formulation 32
9.1 Axiom 1 (Test Category) . . . 32
9.2 Axiom 2 (Behavior Functor) . . . 32
9.3 Axiom 3 (Meaning) . . . 33
9.4 Axiom 4 (Discriminative Equivalence) . . . 33
9.5 Axiom 5 (Verification) . . . 33
9.6 Axiom 6 (Test-Centrism Category) . . . 34
9.7 Yoneda's Theorem in Test-Centrism . . . 34
9.7.1 Theorem . . . 34
9.8 Categorical Interpretation of Understanding . . . 34

10 Internal Logic of the Topos 34
10.1 The Subobject Classifier . . . 35
10.2 Meaning of a Sieve . . . 35
10.3 Meaning of a Proposition . . . 35
10.4 Meaning of Truth . . . 36
10.5 Logical Connectives . . . 36
10.5.1 AND (Conjunction) . . . 36
10.5.2 OR (Disjunction) . . . 36
10.5.3 Implication . . . 36
10.5.4 Negation . . . 36
10.6 Why the Law of Excluded Middle Does Not Hold . . . 37
10.7 Philosophical Interpretation . . . 37
10.8 Correspondence with Philosophy of Science . . . 37

11 Kripke Semantics 38
11.1 Kripke Frame . . . 38

11.2 Forcing Relation . . . 38
11.3 Interpretation of Logical Connectives in Kripke Semantics . . . 38
11.3.1 AND(Conjunction in Kripke Semantics) . . . 38
11.3.2 OR(Disjunction in Kripke Semantics) . . . 38
11.3.3 Implication in Kripke Semantics . . . 39
11.3.4 Negation in Kripke Semantics . . . 39
11.4 Correspondence with the Topos . . . 39
11.5 Philosophical Meaning . . . 39

12 Social Theory 40

13 Political theory or the theory of tolerance 40
13.1 Starting Point . . . 40
13.2 Fundamental Principles . . . 41
13.2.1 Principle 1: Freedom of Thought . . . 41
13.2.2 Principle 2: Evaluation Based on Consequences . . . 41
13.3 Test Structure (Formalization) . . . 41
13.3.1 Two Types of Tests . . . 41
13.4 Regulatory Principle (Core Proposition) . . . 42
13.4.1 TC Regulatory Principle . . . 42
13.5 Three-Layer Structure (Important) . . . 42
13.5.1 Level 1: Thought . . . 42
13.5.2 Level 2: Interpretation . . . 42
13.5.3 Level 3: Action . . . 42
13.6 Definition of Tolerance (TC Version) . . . 43
13.6.1 TC Tolerance . . . 43
13.6.2 Key Implication . . . 43
13.7 Response to Classical Problems . . . 43
13.7.1 Response to Popper's Paradox of Tolerance . . . 43
13.8 Core Insight (Central Thesis) . . . 43
13.9 Advantages . . . 43
13.9.1 Maximum freedom . . . 43
13.9.2 Precise regulation . . . 44
13.9.3 Avoidance of social division . . . 44
13.9.4 Consistency with empiricism . . . 44
13.10 Weaknesses (Important Issues to Address) . . . 44
13.10.1 Validity of prediction . . . 44
13.10.2 Risk threshold . . . 44
13.10.3 Boundary of interpretation . . . 44
13.10.4 Risk of power abuse . . . 44

13.11 One-Sentence Summary 44

14 Conclusion 44

1 Introduction

This paper generalizes the idea of Test-Driven Development (TDD) and proposes **Test-centrism** as a framework for understanding concepts, propositions, and theories.

In TDD, software specifications are described through tests, and program behavior is defined via those tests. This idea extends beyond software development and applies more broadly.

Similar ideas exist in philosophy. In pragmatism, C.S. Peirce stated that **the meaning of a concept lies in its practical consequences**. In the philosophy of science, Karl Popper argued that **scientific theories must be falsifiable**.

Although these perspectives concern different objects, they share a common idea:

```
Whether code, concept, proposition, or theory,
all objects are understood through the tests that distinguish them.
```

This paper generalizes this idea into a framework applicable to:

- concepts
- propositions
- theories

and presents it as **Test-centrism**.

2 TDD

2.1 Test-Driven Development

Test-Driven Development (TDD) is a software development methodology. While the usual process is "specification → implementation → testing," TDD reverses this order to "specification → testing → implementation," writing tests first.

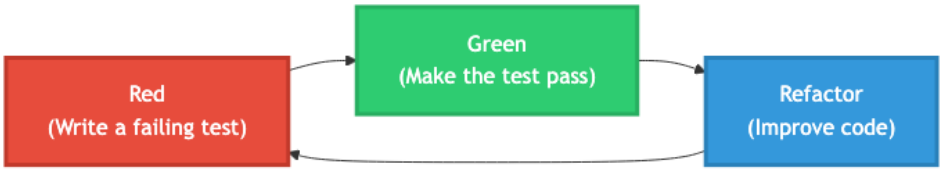
The fundamental principles of TDD are:

> - Write new code only if an automated test has failed
> - Eliminate duplication
>
> Kent Beck, *Test-driven development: by example*, 2003, p.ix

In other words, the first principle is that **don't implement anything unil a test fails**.

The famous TDD mantra is:

```
Red → Green → Refactor
(ibid., p.x)
```



1. Red: Write a failing test
2. Green: Make the test pass with minimal implementation
3. Refactor: Improve the code without changing behavior

A key feature of TDD is that **specifications are expressed in tests**. By doing so, you can first clarify **what** you want to achieve, and then think about **how** to achieve it.

According to Martin Fowler:

> In fact, one of the most useful times to write tests is before I start programming. When I need to add a feature, I begin by writing the test, I'm asking myself what needs to be done to add the function. Writing the test also concentrates me on the interface rather than the implementation(always a good thing). It also means I have a clear point at which I'm done coding–when the test works.
> Martin Fowler, *Refactoring*, 2nd edition, p.86

Freeman and Pryce also point out:

> First, starting with a test means that we have to describe *what* we want to achieve before we consider *how*. This focus helps us maintain the right level of abstraction for the target object. If the intention of the unit test is unclear then we're probably mixing up concepts and not ready to start coding. It also helps us with information hiding as we have to decide what needs to be visible from outside the object.
> Steve Freeman, Nat Pryce, *Growing Object Oriented Software Guided by Tests*, p.57

2.2 Refactoring

In *Refactoring*, Martin Fowler defines refactoring as follows:

> **Refactoring**(noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.
> Fowler, op. cit., p.45

In other words, refactoring is a transformation that **preserves behavior while changing only the internal structure to make the code easier to understand**.

The important point is that **behavioral equivalence is guaranteed by tests**.

As long as the tests pass, the programs are considered **equivalent**.

2.3 What is a Test?

In *The art of software testing*, testing is defined as follows:

> Testing is the process of executing a program with the intent of finding errors.
> *The art of software testing*, p.6

Simply put, testing is **the process of running a program for the purpose of finding errors**.

2.4 Example of TDD

As an example of TDD, let us consider a program that calculates the greatest common divisor (GCD). That is,

```
Given any two natural numbers a and b,
compute their greatest common divisor (GCD).

Input: a, b
Output: gcd
```

There are several algorithms to compute the GCD. The most famous is the Euclidean algorithm, but a simple brute-force approch is also possible.

Here we will construct the program following the TDD methodology.

2.4.1 STEP 1 — The first test

In TDD, we begin by writing a test. Let the first test X_1 be defined as:

- Input:

$$i_1 = (1, 1)$$

- Expected value:

$$o_1 = 1$$

That is, we construct the test

$$X_1 = (i_1,\, o_1) = ((1, 1), 1)$$

```python
# `tests/tdd_gcd_tutorial/gcd1/test_1.py`
from src.tdd_gcd_tutorial.gcd1 import gcd1

def test_t1():
```

```
    assert gcd1(1, 1) == 1
```

Next, we write the minimal implementation that passes this test.

```
# `src/tdd_gcd_tutorial/gcd1.py`
def gcd1(a: int, b: int) -> int:
    # Return 1.
    return 1
```

Since this is a trivial implementation that simply returns `1`, no refactoring is necessary.

2.4.2 STEP 2 — Adding a test

Add the next test X_2.

- Input:

$$i_2 = (3, 1)$$

- Expected value:

$$o_2 = 1$$

That is, the test is

$$X_2 = (i_2,\, o_2) = ((3, 1), 1)$$

At this point, since $gcd1(i_2) = o_2 = 1$, the function `gcd1` **passes** both X_1, X_2. Therefore **no changes are made to the implementation**.

2.4.3 STEP 3 — A failing test

Next, we add

$$X_3 = (i_3, o_3) = ((24, 8), 8)$$

At this point,

$$gcd1(i_3) = 1 \neq 8 = o_3$$

Therefore, the test X_3 **fails**.

```
FAILED tests/tdd_gcd_tutorial/gcd1/test_3.py::test_t3 - assert 1 == 8
```

In TDD, we add the minimal implementation that makes the failing test pass. Therefore we implement the following:

```python
# src/tdd_gcd_tutorial/gcd2.py
def gcd2(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    return b
```

The function `gcd2` **passes** all tests

$$X_1, X_2, X_3.$$

No refactoring is necessary.

2.4.4 STEP 4 — The Problem of Input Order

Add the following test:

$$X_4 = (i_4, o_4) = ((8, 24), 8).$$

At this point, `gcd2` satisfies $gcd2(i_4) \neq o_4$, and therefore **fails**.

```
FAILED tests/tdd_gcd_tutorial/gcd2/test_4.py::test_t4 - assert 24 == 8
```

Therefore, we create the following implementation:

```python
def gcd3(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    c = b
    if a < b:
        c = a
    return c
```

The function `gcd3` **passes** all tests

$$X_1, \ldots, X_4.$$

This implementation is somewhat redundant. Since using the `min` method is clearer, we refactor it as follows:

```python
# src/tdd_gcd_tutorial/gcd3.py
def gcd3(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    return min(a, b)
```

This implementation passes the tests.

2.4.5 STEP 5 — When Division Does Not Complete in One Step

Consider adding the following test:

$$X_5 = (i_5, o_5) = ((24, 16), 8)$$

At this point, `gcd3` satisfies $gcd3(i_5) \neq o_5$, and therefore **fails**.

```
FAILED tests/tdd_gcd_tutorial/gcd3/test_5.py::test_t5 - assert 16 == 8
```

Therefore, we implement a function that performs only one step of the Euclidean algorithm.

```python
# src/tdd_gcd_tutorial/gcd4.py
def gcd4(a: int, b: int) -> int:
    # One step of the Euclidean algorithm.
    # Replace the smaller number with the remainder.
    x = max(a, b)
    y = min(a, b)
    r = x % y
    if r:
        y = r
    return y
```

The function `gcd4` **passes** all tests

$$X_1, \ldots, X_5.$$

Since this implementation is somewhat redundant, we refactor it as follows:

```python
def gcd4(a: int, b: int) -> int:
    # One step of the Euclidean algorithm.
    # Replace the smaller number with the remainder.
    a, b = max(a, b), min(a, b)
    return a % b if a % b else b
```

This implementation passes the tests.

2.4.6 STEP 6 — Coprime case

Finally, add

$$X_6 = (i_6, o_6) = ((24, 7), 1)$$

At this point, `gcd4` satisfies $gcd4(i_6) \neq o_6$, and therefore **fails**.

```
FAILED tests/tdd_gcd_tutorial/gcd4/test_6.py::test_t6 - assert 3 == 1
```

Accordingly, we implement the full Euclidean algorithm.

```python
# src/tdd_gcd_tutorial/gcd5.py
def gcd5(a: int, b: int) -> int:
    # Straightforward Euclidean algorithm. Time complexity: O(log n).
    while True:
        if b == 0:
            return a
        else:
            r = a % b
            a = b
            b = r
```

The function `gcd5` **passes** all tests

$$X_1, \ldots, X_6.$$

This completes the implementation of the Euclidean algorithm.

Although `gcd5` is one implementation of the Euclidean algorithm, it is somewhat verbose. Accordingly, the following alternative implementations such as `gcd6` and `gcd7` also pass all tests as does `gcd5`.

```python
# src/tdd_gcd_tutorial/gcd6.py
def gcd6(a: int, b: int) -> int:
    # Overly short Euclidean algorithm. Time complexity: O(log n).
    return a if b == 0 else gcd6(b, a % b)

# src/tdd_gcd_tutorial/gcd7.py
def gcd7(a: int, b: int) -> int:
    # Human-readable Euclidean algorithm. Time complexity: O(log n).
    while b !=0 :
        a, b = b, a % b
    return a
```

Furthermore, the following `gcd8` provides an example of a naive division search (brute-force) approach.

```python
def gcd8(a: int, b: int) -> int:
    # Naive division search (brute-force). Time complexity: O(n).
    m = min(a, b)
    for i in range(m, 0, -1):
        if a % i == 0 and b % i == 0:
            return i
```

These are candidates for refactoring; however, each has its own characteristics. `gcd6` is too concise and difficult for humans to read, while `gcd8` has a different time complexity from `gcd5`. Therefore, the most appropriate refactoring is `gcd7`.

3 Pragmatic Maxim

Charles Sanders Peirce, one of the founders of pragmatism, states the meaning of an idea as follows:

> Consider what effects, that might conceivably have practical bearings, we conceive the object of our conception to have. Then, our conception of these effects is the whole of our conception of the object.
> CP 5.402

This statement is called the **Pragmatic Maxim**. Peirce argued that it is a logical principle necessary for scientific inquiry(the exact reference needs to be verified; he may have been referring to abduction). The maxim can be paraphrased as:

```
The meaning of an idea consists in its observable effects
```

To put it another way, to understand a concept is to know what results it would produce under various circumstances.

For example, consider the concept "hard" (CP 5.403, CP 5.467). The meaning of this concept is clarified by **observable effects**, such as:

- It produces a sound when struck
- It doesn't deform when pressed
- It is difficult to scratch with a knife

In other words, the meaning of a concept is understood through the tests that distinguish it.

3.1 Encoding the Pragmatic Maxim in TypeScript

This section is intended only for readers familiar with TypeScript; others may skip it. It is merely a digression—a kind of interlude—and not part of the main line of argument, so it can be set aside without loss.

Let us once again quote the pragmatic maxim mentioned earlier.

> Consider what effects, that might conceivably have practical bearings, we conceive the object of our conception to have. Then, our conception of these effects is the whole of our conception of the object.

Try to express the pragmatic maxim in TypeScript. It might then be written as follows:

```typescript
// pragmatism.ts

// Consider what effects, that might conceivably have practical bearings,
// we conceive the object of our conception to have.
```

```
// Then, our conception of these effects is
// the whole of our conception of the object.
// CP 5.402
// concept = interface
// object of conception = class + instance
// practical effects = the results of operations performed on an instance
// meaning of concept = the totality of observable behavior

// ==============================
// 1) Concept = Interface
// ==============================

// Concept of Hard
interface ConceptOfHard {
    press(force: number): string
    scratch(tool: string): string
}

// Concept of Soft
interface ConceptOfSoft {
    press(force: number): string
    scratch(tool: string): string
}

// Concept of Heavy
interface ConceptOfHeavy {
    lift(force: number): string
    drop(): string
}
```

```
// ==============================
// 2) Object of Conception = Class + Instance
// ==============================

// Hard Object
class Steel implements ConceptOfHard {
    press(force: number): string {
        return 'does not deform'
    }
```

```
    scratch(tool: string): string {
        return 'not scratched'
    }
}

// Soft Object
class Sponge implements ConceptOfSoft {
    press(force: number): string {
        return 'deforms'
    }
    scratch(tool: string): string {
        return 'scratched eaily'
    }
}

// Heavy Object
class LeadBlock implements ConceptOfHeavy {
    lift(force: number): string {
        return force < 100 ? 'cannot lift': 'lifted'
    }
    drop(): string {
        return 'makes loud thud'
    }
}

// instantiating objects (object of conception)
const steel = new Steel()
const sponge = new Sponge()
const leadBlock = new LeadBlock()
```

```
// ==============================
// 3) Practical Effects (operational results)
// ==============================

console.log("=== Practical Effects ===")

// Hardness
console.log("Steel press 100:", steel.press(100))            // does not deform
console.log("Steel scratch knife:", steel.scratch("knife")) // not scratched
```

```
console.log("Sponge press 100:", sponge.press(100))         // deforms
console.log("Sponge scratch knife:", sponge.scratch("knife")) // scratched easily

// Heaviness
console.log("Lead lift 50:", leadBlock.lift(50))            // cannot lift
console.log("Lead lift 150:", leadBlock.lift(150))         // lifted
console.log("Lead drop:", leadBlock.drop())                 // makes loud thud
```

```
// ==============================
// 4) Meaning of Concept
// ==============================

type Meaning<T> = { [K in keyof T]: string }

const meaningSteel: Meaning<ConceptOfHard> = {
  press: steel.press(100),
  scratch: steel.scratch("knife")
}

const meaningSponge: Meaning<ConceptOfSoft> = {
  press: sponge.press(100),
  scratch: sponge.scratch("knife")
}

const meaningLead: Meaning<ConceptOfHeavy> = {
  lift: leadBlock.lift(150),
  drop: leadBlock.drop()
}

console.log("=== Meaning of Concept ===")
console.log("Steel meaning:", meaningSteel)
console.log("Sponge meaning:", meaningSponge)
console.log("Lead meaning:", meaningLead)
```

```
=== Practical Effects ===
Steel press 100: does not deform
Steel scratch knife: not scratched
Sponge press 100: deforms
```

```
Sponge scratch knife: scratched eaily
Lead lift 50: cannot lift
Lead lift 150: lifted
Lead drop: makes loud thud
=== Meaning of Concept ===
Steel meaning: { press: 'does not deform', scratch: 'not scratched' }
Sponge meaning: { press: 'deforms', scratch: 'scratched eaily' }
Lead meaning: { lift: 'lifted', drop: 'makes loud thud' }
```

Peirce terminology	TypeScript code	Description
concept	ConceptOfHard / ConceptOfSoft / ConceptOfHeavy	The specification of notions such as "hardness," "softness," and "heaviness"
object of conception	steel / sponge / leadBlock	Concrete entities that exist (instances)
practical effects	results of press(), scratch(), lift()	The behaviors obtained by applying operations
meaning of concept	Meaning<T> object	A set of observable behaviors

3.1.1 Why "hard" should be an interface, not a class

■The nature of a concept According to Peirce, a **concept is a specification of the observable behavior an object may exhibit**. The concept of "hardness," for example: - defines what counts as hardness (e.g., resists deformation under pressure, is difficult to scratch) - does **not** determine which specific things are hard In other words, "hardness" is an **abstract set of rules or specifications**, not a physical entity.

■In OOP terms

- **interface = specification / abstraction**
  - defines what must be implemented
  - does not include concrete behavior itself
- **class = implementation / concretization**
  - realizes the specification with concrete behavior
  - can be instantiated as objects

Peirce	OOP	Description
concept (e.g., hardness)	interface ConceptOfHard	Defines a specification of observable "hardness" behavior
object of conception	class Steel / instance steel	A concrete object that can actually be pressed or scratched

■Key reasons

- **Level of abstraction**
  - Concepts are abstract → interface
  - Classes are concrete implementations → objects of conception
- **Focus on behavioral specification**
  - The meaning of "hard" lies in behavior (e.g., "does not deform under pressure")
  - A class realizes such behavior, but is not the concept itself
- **In terms of Peirce's pragmatic maxim**
  - The meaning of a concept is the totality of its observable effects
  - A class corresponds to an object that produces such effects
  - The concept itself is still at the level of a **specification or rule set**, i.e., an interface

■Conceptual diagram

Peirce	OOP
concept (e.g., hardness)	interface ConceptOfHard
object of conception	steel = new Steel()
practical effects	steel.press(), steel.scratch()
meaning of concept	the totality of observed results of actions

- Concepts such as "hardness" are **sets of specifications or rules**, not concrete entities.
- Therefore, they correspond to an **interface rather than a class**.
- A class is the concrete realization of a concept, not the concept itself.

4 Falsifiability

Karl Popper proposed **falsifiability** as a criterion for scientific theories. According to Popper, a scientific theory is

```
a theory that can potentially be shown to be false by observation
```

For example, consider the following universal proposition:

```
All swans are white
```

This proposition is falsified if a black swan is observed. In other words, a scientific theory, as a system of universal propositions, is

```
a theory that can be shown to be false
if a certain observation occurs
```

In this sense, scientific theories always contain **testable predictions**.

5 Toward a Formalization of Test-Centrism: Some Examples

5.1 Greatest Common Divisor

Let us consider the programs `gcd1` through `gcd8` for computin the greatest common divisor, as introduced in the TDD example.

5.1.1 Behavioral Function

The function `gcd` takes a pair of natural numbers as input and return a natural number as output. That is, letting the **input set** be I and the **output set** be O, we have

$$I = \mathbb{N}^2$$
$$O = \mathbb{N}$$

For each input value $i \in I$, there exists an output

$$gcd(i) \in O.$$

Therefore, we define the **behavioral function** of `gcd` as

$$\text{behavior}_{gcd} : I \to O.$$

For example, given the input i_1 from test X_1, the behavioral function behavior_{gcd1} of `gcd1` yields

$$\text{behavior}_{gcd1}(i_1) = 1$$

This means:

```
the program can produce a response for the given input
```

5.1.2 Tests

In TDD, each test is typically associated with an expected value. However, in practice, there are cases where the expected value is not known in advance. Therefore, we classify tests into the following two types:

- Observed test
- Open test

On the one hand, an **observed test** is a test in which both the input and its corresponding expected value are given.

$$X_{ob} = (i,\, o) \in I \times O$$

On the other hand, an **open test** is a test in which the expected value (observed output) is not given. Here, we denote an undefined expected value by $\perp$.

$$X_{op} = (i,\, \perp) \in I \times \{\, \perp \,\}$$

Therefore, a test is given by

$$X = (i, o) \in I \times (O \cup \{\perp\})$$

Note that given an observed test $X_{ob} = (i, o)$, we can construct the corresponding open test X_{op} by removing the expected value:

$$(i,\, \perp).$$

5.1.3 Test Sequences

In general, tests have an order. Therefore, we treat them not as a set but as a **test sequence**. A test sequence $\mathbb{X}$ is an ordered list of a finite number of tests. That is,

$$\mathbb{X} = (X_1, X_2, \ldots, X_n)$$

where, $X_j \in I \times (O \cup \perp)$. For example, consider the following test sequence:

$$\mathbb{X} = ((i_1,\, o_1),\, (i_2,\, o_2),\, (i_3,\, o_3),\, (i_4,\, \perp)).$$

In this case, the first through third elements are observed tests, while the fourth is an open test.

We define a test sequence for `gcd`, $\mathbb{X} = (X_1,\, X_2,\, X_3,\, X_4,\, X_5,\, X_6)$, as follows:

Test	Input i	Output o
X_1	(1, 1)	1
X_2	(3, 1)	1
X_3	(24, 8)	8
X_4	(8, 24)	8
X_5	(24, 16)	8
X_6	(24, 7)	1

5.1.4 Test Results

■Discrimination When each `gcd` is executed, the following results are obtained.

	i_1: (a,b) = (1, 1)	i_2: (a,b) = (3, 1)	i_3: (a,b) = (24, 8)	i_4: (a,b) = (8, 24)	i_5: (a,b) = (24, 16)	i_6: (a,b) = (24, 7)
gcd1	1	1	1	1	1	1
gcd2	1	1	8	24	16	7
gcd3	1	3	8	8	16	7
gcd4	1	1	8	8	8	3
gcd5	1	1	8	1	8	1
gcd6	1	1	8	1	8	1
gcd7	1	1	8	1	8	1
gcd8	1	1	8	1	8	1

From this, we can observe the following:

1. Since `gcd5` and `gcd6` produce the same outputs for the test sequence $\mathbb{X}$, they cannot be distinguished based on their behavior. In this case, we shall say that `gcd5` is **discriminative equivalent** to `gcd6` with respect to the test sequence $\mathbb{X}$. This may also be called **behavioral equivalence**. Similarly, `gcd5` is also discriminative equivalent to `gcd7` and `gcd8` with respect to $\mathbb{X}$.
2. `gcd1` and `gcd2` produce different outputs for test X_3, therefore they can be distinguished by their behavior. That is , `gcd1` is not discriminative equivalent to `gcd2` with respect to the test sequence $\mathbb{X}$. Similarly `gcd1` is not discriminative equivalent to `gcd3`, `gcd4` or `gcd5`.

■Verification When the tests are executed, the following results are obtained:

	X_1: (i_1, o_1) = ((1, 1), 1)	X_2: (i_2, o_2) = ((3, 1), 1)	X_3: (i_3, o_3) = ((24, 8), 8)	X_4: (i_4, o_4) = ((8, 24), 8)	X_5: (i_5, o_5) = ((24, 16), 8)	X_6: (i_6, o_6) = ((24, 7), 1)
gcd1	True	True	False	False	False	True
gcd2	True	True	True	False	False	False
gcd3	True	True	True	True	False	False
gcd4	True	True	True	True	True	False
gcd5	True	True	True	True	True	True
gcd6	True	True	True	True	True	True
gcd7	True	True	True	True	True	True
gcd8	True	True	True	True	True	True

X_1: (i_1, o_1) = ((1, 1), 1)	X_2: (i_2, o_2) = ((3, 1), 1)	X_3: (i_3, o_3) = ((24, 8), 8)	X_4: (i_4, o_4) = ((8, 24), 8)	X_5: (i_5, o_5) = ((24, 16), 8)	X_6: (i_6, o_6) = ((24, 7), 1)

From this, we can observe the following:

1. `gcd5` produces outpts that match the expected values for all tests in the test sequence $\mathbb{X}$. In this case, we shall say that `gcd5` is **verified** by the test sequence $\mathbb{X}$. Smilarly, `gcd6`, `gcd7` and `gcd8` are also verified by $\mathbb{X}$.
2. Both `gcd5` and `gcd6` are verified by the test sequence $\mathbb{X}$. In this case, we shall say that `gcd5` is **verification equivalent** to `gcd6` with respect to $\mathbb{X}$. This may also be called **specification equivalence**. Similarly, `gcd5` is verification equivalent to `gcd7` and `gcd8` with respect to $\mathbb{X}$.
3. `gcd1` fails to match the expected valuee in at least one test in the test sequence $\mathbb{X}$. For example, `gcd1` does not match the expected value (`False`) in test X_3. In this case, we shall say that `gcd1` is **falsified** by the test sequence $\mathbb{X}$. Similarly, `gcd2`, `gcd3`, and `gcd4` are falsified by $\mathbb{X}$.
4. Furthermore, comparing with the previously discussed Discrimination, we observe that if any two implementations of `gcd` are verification equivalent with respect to the test sequence $\mathbb{X}$, then they are also discriminative equivalent with respect to $\mathbb{X}$.

5.2 Example of Pragmatism

Consider the concepts of "hard" and "soft." To understand these concepts, we can think of the following test:

- Scratch with a knife

The pragmatic maxim can be regarded as a principle that satisfies the following: the object of a concept is **discriminable** by tests, and the meaning of a concept is the behavior it exhibits in response to those tests.

```
Meaning = behavior exhibited in tests
```

Based on the results of these tests, "hard" and "soft" are distinguished as concepts. That is,

```
Concepts are understood through the tests that distinguish them
```

What the pragmatic maxim directly asserts is that concepts must be discriminable. That is, by using a concept, one must be able to distinguish what differences arise under different conditions.

However, at this stage, the actual outcome is not yet determined. By applying a concept, we obtain predictions about what will occur, but whether those predictions are correct remains unknown.

Whether those predictions are correct is determined through experiments or observations. Only at this point are the predictions based on the concept verified.

This is why experiments are indispensable in scientific inquiry. A concept must be discriminable in

order to be verified through experiments. The pragmatic maxim presents this discriminability as a condition for the meaning of a concept.

5.3 Example of a Scientific Theory

A scientific theory, too, is something that is understood through testing. For example, consider the following proposition P P:

Scientific theories are also something that is understood through tests. For example, consider the following proposition P:

```
All swans are white
```

This proposition can be examined by observing swans. For instance, we can consider the following test:

```
Observe a swan and check its color
```

Suppose that, as a result of observations carried out so far, we have obtained an observed test sequence

$$\mathbb{X}_n = (X_1, \ldots, X_n)$$

where each test X_i consists of observing a swan and checking its color.

If all swans observed so far are white, then the proposition P is not inconsistent with the observed results. That is, the proposition P is verified by the test sequence $\mathbb{X}_n$.

Next, consider the case where observations continue further. Let us consider a test sequence $\mathbb{X}_N$ in which the first $N-1$ tests are observed tests and the N-th test is an open test:

$$\mathbb{X}_N = (X_1, \ldots, X_n, X_{n+1}, \ldots, X_{N-1}, X_N)$$

where $X_i \in I \times O\,(i = 1, \ldots, N-1)$ and $X_N \in I \times \{\perp\}$.

The proposition P provides the following prediction for the input i_N of the open test X_N:

$$\text{behavior}_P(i_N) = \text{the swan}\, i_N \text{ is white}$$

If, in this observation, a non-white swan (for example, a black swan) is observed, then

$$\text{behavior}_P(i_N) \neq o_N$$

and the proposition P is falsified by the test sequence $\mathbb{X}_N$.

In this way, scientific theories can be falsified by observation. In other words, a scientific theory is characterized by the tests that could potentially falsify it.

```
A theory is characterized by the tests that could potentially falsify it
```

In this sense, a scientific theory always contains predictions about phenomena that have not yet been observed.

6 Test-Centrism

6.1 Basic Idea

We classify the subject into three types:

- Concept
- Proposition
- Theory

The basic stance of Test-centrism is as follows:

```
Objects are understood through tests
```

Here, a "test" refers to a procedure in which some operation or observation is performed on an object and a result is obtained.

In the case of a concept, a test is an operation that examines what observable results the concept produces.

In the case of a proposition, a test is an observation that checks whether the proposition holds.

In the case of a theory (a system of propositions), a test is a procedure that compares the predictions given by the theory with observations.

In this way, concepts, propositions, and theories are all objects whose meaning or content is clarified through tests.

6.2 The Role of Tests

Tests have two roles: **tests for construction** and **tests for understanding**.

6.2.1 Tests for Construction

Tests for construction are the tests used in TDD. That is, "specification → test → implementation."

6.2.2 Tests for Understanding

Tests for understanding are tests performed to understand an object (concept, proposition, or theory). That is, "object → test → behavior." To understand any object is to be able to construct the tests that distinguish it.

7 Formalization

We formalize Test-Centrism.

7.1 Axiom 1 (Objects)

Let the set of objects be

$$\mathcal{T}.$$

Each object $T \in \mathcal{T}$ belongs to one of the following types:

- Concept
- Proposition
- Theory

7.2 Axiom 2 (Behavioral Function)

There exist an input set I and an output set O. Each object

$$T \in \mathcal{T}$$

has a behavioral function

$$\text{behavior}_T : I \to O.$$

7.3 Axiom 3 (Tests)

A test X for an object T is given by

$$(i,\, o) \in I \times (O \cup \{\perp\})$$

where:

- $(i,\, o) \in I \times O$ is called an observed test
- $(i,\, \perp) \in I \times \{\perp\}$ is called an open test.

An open test represents a test for which the actual observation has not yet been determined.

7.4 Axiom 4 (Test Sequence)

A test sequence $\mathbb{X}$ is finite ordered sequence of tests:

$$\mathbb{X} = (X_1,\, \ldots, X_n)$$

where each X_i is a test satisfying

$$X_i \in I \times (O \cup \{\perp\}).$$

The set of inputs of the test sequence $\mathbb{X}$ is defined by

$$\mathit{inputs}(\mathbb{X}) = \{\, p_i : X_i \to I \text{ is the projection} \mid X_i \in \mathbb{X} \,\}.$$

7.5 Axiom 5 (Meaning)

The **meaning** of an object T with respect to a test sequence $\mathbb{X}$, denoted by $\text{Meaning}_{\mathbb{X}}(T)$, is defined as

$$\text{Meaning}_{\mathbb{X}}(T) = (\text{behavior}_T(i_1), \ldots, \text{behavior}_T(i_n))$$

where $i_j \in \mathit{inputs}(\mathbb{X})$.

7.6 Axiom 6 (Verifiability)

A test sequence $\mathbb{X}$ is said to be **verifiable** if every test $X_i \in \mathbb{X}$ is an observed test.

7.7 Axiom 7 (Discriminative Equivalence)

Objects $T, S \in \mathcal{T}$ are said to be **discriminative equivalent** with respect to a test sequence $\mathbb{X}$ if

$$\forall i \in \mathit{inputs}(\mathbb{X}) : \text{behavior}_T(i) = \text{behavior}_S(i).$$

In this case, we write

$$T \sim_{\mathbb{X}} S.$$

This means:

```
they cannot be distinguished by the given test sequence
```

7.8 Axiom 8 (Verification Equivalence)

An object $T \in \mathcal{T}$ is said to be **verified** by a verifiable test sequence $\mathbb{X}$, written

$$T \vDash \mathbb{X}$$

if for every test $X = (i,\, o)$ in the test sequence $\mathbb{X}$,

$$\text{behavior}_T(i) = o.$$

Objects $T, S \in \mathcal{T}$ are said to be **verification equivalent** with respect to a test sequence $\mathbb{X}$ if

$$T \vDash \mathbb{X} \quad \wedge \quad S \vDash \mathbb{X},$$

In this case, we write

$$T \approx_{\mathbb{X}} S.$$

This means:

```
both pass the given test sequence
```

If objects $T, S \in \mathcal{T}$ are verification equivalent, then they are discriminative equivalent. That is,

$$T \approx_{\mathbb{X}} S \Rightarrow T \sim_{\mathbb{X}} S.$$

7.9 Axiom 9 (Understanding)

An agent is said to **understand** an object $T \in \mathcal{T}$ with respect to a test sequence $\mathbb{X}$ if, for any input $i \in \mathit{inputs}(\mathbb{X})$, it has the ability to predict (or determine)

$$\text{behavior}_T(i).$$

```
Understanding = the ability to respond to tests
```

7.10 Axiom 10 (Complexity)

A complexity function is defined on $\mathcal{T}$:

$$C : \mathcal{T} \to \mathbb{R}_{\geq 0}.$$

7.11 Axiom 11 (Refactoring)

For objects $T, S \in \mathcal{T}$, S is called a **refactoring** of T if

$$T \approx_{\mathbb{X}} S \wedge C(S) \leq C(T).$$

8 Examples

8.1 Greatest Common Divisor: An Example of Verifiable Objects

We regard `gcd` as a proposition.

- Set of objects:

$$\mathcal{T}_{\text{gcd}} = \{\, \text{gcd1, gcd2, gcd3, gcd4, gcd5, gcd6, gcd7, gcd8} \,\}$$

- Verifiable test sequence:

$$\mathbb{X} = (X_1,\, X_2,\, X_3,\, X_4,\, X_5,\, X_6)$$

Each test has the form

$$X_k = (i_k,\, o_k)$$

where, input: $i_k = (i_{k_1},\, i_{k_2}) \in \mathbb{N}^2$ and observation: $o_k \in \mathbb{N}$.

- Verifiability:

All objects

$$T \in \mathcal{T}_{\text{gcd}}$$

are verifiable with respect to the test sequence $\mathbb{X}$.

- Discriminative equivalence:

$$\begin{gathered}
\text{gcd1} \nsim_{\mathbb{X}} \text{gcd2} \\
\text{gcd1} \nsim_{\mathbb{X}} \text{gcd3} \\
\text{gcd1} \nsim_{\mathbb{X}} \text{gcd4} \\
\text{gcd1} \nsim_{\mathbb{X}} \text{gcd5} \\
\text{gcd2} \nsim_{\mathbb{X}} \text{gcd3} \\
\text{gcd2} \nsim_{\mathbb{X}} \text{gcd4} \\
\text{gcd2} \nsim_{\mathbb{X}} \text{gcd5} \\
\text{gcd3} \nsim_{\mathbb{X}} \text{gcd4} \\
\text{gcd3} \nsim_{\mathbb{X}} \text{gcd5} \\
\text{gcd5} \sim_{\mathbb{X}} \text{gcd6} \\
\text{gcd5} \sim_{\mathbb{X}} \text{gcd7} \\
\text{gcd5} \sim_{\mathbb{X}} \text{gcd8}
\end{gathered}$$

- Verification equivalence:

$$
\begin{aligned}
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd2} \\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd3} \\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd4} \\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd5} \\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd3} \\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd4} \\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd5} \\
&\text{gcd3} \not\approx_{\mathbb{X}} \text{gcd4} \\
&\text{gcd3} \not\approx_{\mathbb{X}} \text{gcd5} \\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd6} \\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd7} \\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd8}
\end{aligned}
$$

- Meaning:

$$
\begin{aligned}
\text{Meaning}_{\mathbb{X}}[\text{gcd5}] &= (\text{behavior}_{\text{gcd5}}(i_1), \ldots, \text{behavior}_{\text{gcd5}}(i_6)) \\
&= (1,\ 1,\ 8,\ 1,\ 8,\ 1) \\
&= \text{Meaning}_{\mathbb{X}}[\text{gcd6}] = \text{Meaning}_{\mathbb{X}}[\text{gcd7}] = \text{Meaning}_{\mathbb{X}}[\text{gcd8}]
\end{aligned}
$$

That is, `gcd5`, `gcd6`, `gcd7` and `gcd8` have the same meaning with reespect to the test sequence.

- Understanding:

To understand `gcd5` means to have the ability to predict, for any input $i \in \mathit{inputs}(\mathbb{X})$,

$$\text{behavior}_{\text{gcd5}}(i).$$

- Refactoring:

The transformation from `gcd5` to `gcd7` is a refactoring.

$$\text{gcd5} \rightarrow \text{gcd7}.$$

8.2 The Pragmatic Maxim: Hardness

- Set of objects::

$$\mathcal{T}_{\text{hs}} = \{\ \text{hard}\ ,\ \text{soft}\ \}$$

- Test sequence:

$$\mathbb{X} = ((\text{ knife-edge test}, \perp))$$

This test sequence consists only of an open test.

- Discriminability (Pragmatic Maxim):

According to the pragmatic maxim, the meaning of a concept is given by its **conceivable effects**. "Hard" is defined by its behavior when scratched with the edge of a knife.

$$\begin{aligned} \text{behavior}_{\text{hard}}(\text{knife-edge test}) &= \text{resist} \\ \text{behavior}_{\text{soft}}(\text{knife-edge test}) &= \text{scratch} \end{aligned}$$

- Discriminative equivalence:

$$\text{behavior}_{\text{hard}}(\text{knife-edge test}) = \text{resist} \neq \text{scratch} = \text{behavior}_{\text{soft}}(\text{knife-edge test})$$

Therefore,

$$\text{hard} \nsim_{\mathbb{X}} \text{soft}$$

and the two are distinguishable by this open test.

- Meaning:

$$\begin{aligned} \text{Meaning}_{\mathbb{X}}[\text{hard}] &= (\text{behavior}_{\text{hard}}(\text{knife-edge test})) = (\text{resist}) \\ \text{Meaning}_{\mathbb{X}}[\text{soft}] &= (\text{behavior}_{\text{soft}}(\text{knife-edge test})) = (\text{scratch}) \end{aligned}$$

No observations are introduced here. Therefore, this is **an example of discrimination, not verification**. This example formulates the pragmatic maxim—"the meaning of a concept is determined by its practical effects"—within Test-centrism.

8.3 Scientific Theory: Falsifiability

Next, consider a scientific theory.

- Set of objects:

$$\mathcal{T}_{\text{swan}} = \{\, T \,\}$$

where T is the universal proposition: "All swans are white."

- Test sequence:

$$\mathbb{X} = ((i_1, \text{white}), (i_2, \text{white}), \ldots, (i_{n-1}, \text{white}), (i_n, \perp))$$

Assume that the first $n-1$ are observed tests and the n-th is an open test. The testing procedure is:

```
Observe a swan and check its color
```

The proposition T predicts, for all inputs $i \in \mathit{inputs}(\mathbb{X})$,

$$\text{behavior}_T(i) = \text{white}.$$

- Verifiability

Suppose that in the n-th open test, an observation o_n is obtained. If

$$o_n = \text{white}$$

holds, then for all tests in the sequence $\mathbb{X}$ we have $\text{behavior}_T(i) = o_i$, and thus

$$T \vDash \mathbb{X}.$$

In this case, the proposition T is verified by the test sequence $\mathbb{X}$.

- Falsifiability

If the observation in the open test is

$$o_n = \text{black}$$

then

$$\text{behavior}_T(n_i) \neq o_n.$$

Therefore,

$$T \nvDash \mathbb{X}.$$

In this case, the proposition T is falsified by the test sequence $\mathbb{X}$. Thus, a scientific theory is falsifiable in the sense that

```
there is the possibility of observations that do not match the theory's predictions
```

In Test-centrism, this structure is expressed as:

```
falsification = mismatch between behavior and observation
```

9 Categorical Formulation

9.1 Axiom 1 (Test Category)

Define the test category

$$\mathcal{X}$$

as follows:

- Objects

```
finite test sequences
```

$$\mathbb{X} = ((i_1, o_1), \ldots, (i_n, o_n))$$

where $i_k \in I$ and $o_k \in O \cup \{\perp\}$.

- Morphisms

$$\mathbb{X} \to \mathbb{Y}$$

are given by

```
prefix extension
```

that is,

$$\mathbb{X} \preceq \mathbb{Y}$$

(where $\mathbb{X}$ is a prefix subsequence of $\mathbb{Y}$).
This means:

```
tests are extended in an ordered manner
```

9.2 Axiom 2 (Behavior Functor)

For each object $T \in \mathcal{T}$, define the behavior functor

$$F_T : \mathcal{X}^{op} \to \mathbf{Set}$$

by

$$F_T(\mathbb{X}) = \{f : \mathit{inputs}(\mathbb{X}) \to O \mid f(i) = \text{behavior}_T(i)\}.$$

This is the graph of the behavior over test sequences.

- Action on morphisms

For

$$\mathbb{X} \preceq \mathbb{Y}$$

we define

```
restriction (restriction map)
```

This means:

```
as the number of tests increases, the observed behavior also increases
```

9.3 Axiom 3 (Meaning)

The categorical meaning of an object $T \in \mathcal{T}$ is

$$\text{Meaning}(T) = F_T.$$

That is,

```
meaning = behavior presheaf
```

9.4 Axiom 4 (Discriminative Equivalence)

Objects $T, S \in \mathcal{T}$ are said to be discriminatively equivalent if

$$F_T = F_S.$$

That is,

$$\forall i \in I, \quad \text{behavior}_T(i) = \text{behavior}_S(i).$$

This means:

```
indistinguishable with respect to all tests
```

9.5 Axiom 5 (Verification)

An object $T \in \mathcal{T}$ is said to satisfy a test sequence $\mathbb{X}$, denoted by $T \vDash \mathbb{X}$, if for all k,

$$o_k \neq \bot \Rightarrow \text{behavior}_T(i_k) = o_k.$$

This means:

```
agreement is required only for tests with observations
```

9.6 Axiom 6 (Test-Centrism Category)

Define the Test-Centrism category (TC category) by

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}.$$

This is a

```
presheaf topos
```

9.7 Yoneda's Theorem in Test-Centrism

By the Yoneda lemma,

$$\mathrm{Nat}(y(\mathbb{X}), F_T) \cong F_T(\mathbb{X})$$

holds. Here, $y(\mathbb{X}) = \mathrm{Hom}(-, \mathbb{X})$, and Nat denotes the set of natural transformations.

9.7.1 Theorem

An object T is completely determined by its behavior functor F_T. That is,

$$F_T = F_S \iff T \sim S \iff \forall i \in I, \quad \mathrm{behavior}_T(i) = \mathrm{behavior}_S(i).$$

This means:

```
object = the totality of its responses to tests
```

9.8 Categorical Interpretation of Understanding

To understand an object $T \in \mathcal{T}$ with respect to a test sequence $\mathbb{X}$ is to provide

$$F_T(\mathbb{X}).$$

This means:

```
being able to determine the responses to a given test sequence
```

10 Internal Logic of the Topos

The Test-Centrism category

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}$$

is a presheaf topos. Therefore, its internal logic is **intuitionistic logic**.

10.1 The Subobject Classifier

The truth value object

$$\Omega$$

is the subobject classifier.
In a presheaf topos,

$$\Omega(\mathbb{X})$$

is a sieve on $\mathbb{X}$.

10.2 Meaning of a Sieve

Here, a sieve is

```
a set of morphisms into 𝕏
```

which, in this setting, corresponds to

```
a set of prefixes (sub-test sequences) of 𝕏
```

Moreover, a sieve is

```
closed under prefixes
(sieve = future-closed)
```

That is,

```
if a property holds for a test sequence,
then it also holds for all its prefixes
```

10.3 Meaning of a Proposition

The truth value of a proposition P is an element of

$$\Omega(\mathbb{X})$$

that is, a sieve.
This means:

```
a set of test sequences on which P holds,
and continues to hold under all their extensions
```

The key point is:

```
stability under extension of tests
```

10.4 Meaning of Truth

A proposition P is said to be true on a test sequence $\mathbb{X}$ if

```
it belongs to a sieve containing 𝕏
```

This means:

```
P holds on 𝕏,
and remains valid under all subsequent extensions of tests
```

10.5 Logical Connectives

10.5.1 AND (Conjunction)

$$P \wedge Q$$

```
the intersection of the test sequences satisfying both conditions
```

10.5.2 OR (Disjunction)

$$P \vee Q$$

```
P ∨ Q holds on the current test sequence if either P or Q holds
```

Here, it is not merely that "one or the other holds"; it must be guaranteed to persist under extensions.

10.5.3 Implication

$$P \Rightarrow Q$$

```
for any test extension,
if P holds then Q also holds
```

This is crucial and represents:

```
stable inference
```

10.5.4 Negation

$$\neg P$$

is defined as

$$P \Rightarrow \bot$$

and means:

```
P does not hold under any test extension
```

10.6 Why the Law of Excluded Middle Does Not Hold

In general,

$$P \vee \neg P$$

does not hold.

The reason is that there exist states in which

```
not enough tests have yet been performed
```

In other words,

```
neither P nor ¬P has been determined
```

This corresponds to

```
an intermediate state of scientific inquiry
```

10.7 Philosophical Interpretation

The internal logic of Test-centrism is

```
test-dependent intuitionistic logic
```

In this setting: Here:

- Truth = validity that remains unbroken under all future tests
- Proposition = a property dependent on test sequences

10.8 Correspondence with Philosophy of Science

This structure aligns with:

- Popper's falsificationism
- Peirce's pragmatism

In particular, the following interpretation holds:

```
negation = falsifiability
```

- Test sequences = accumulation of knowledge
- Extensions = addition of experiments
- Stability = resistance to refutation

11 Kripke Semantics

The internal logic of Test-centrism can also be interpreted as **Kripke semantics**.

11.1 Kripke Frame

Let a Kripke frame be given by

$$(\mathcal{X}, \preceq).$$

- Worlds: test sequences $\mathbb{X}$
- Accessibility relation: prefix relation $\preceq$

```
as tests are extended, we move forward into the future
```

11.2 Forcing Relation

For a proposition P, we define

$$\mathbb{X} \Vdash P$$

to mean "P holds at $\mathbb{X}$."
The crucial condition here is

```
monotonicity
```

$$\mathbb{X} \preceq \mathbb{Y}, \quad \mathbb{X} \Vdash P \Rightarrow \mathbb{Y} \Vdash P.$$

That is,

```
once a proposition holds,
it continues to hold even as tests increase
```

11.3 Interpretation of Logical Connectives in Kripke Semantics

In Kripke semantics, logical connectives are interpreted as follows.

11.3.1 AND(Conjunction in Kripke Semantics)

$$\mathbb{X} \Vdash P \wedge Q \iff \mathbb{X} \Vdash P \text{ and } \mathbb{X} \Vdash Q$$

11.3.2 OR(Disjunction in Kripke Semantics)

$$\mathbb{X} \Vdash P \vee Q$$

means

```
P ∨ Q holds on the current test sequence
if either P or Q holds
```

However, this validity must be stable with respect to future extensions.

11.3.3 Implication in Kripke Semantics

$$\mathbb{X} \Vdash P \Rightarrow Q$$

means

$$\forall \mathbb{Y} \succeq \mathbb{X}, \quad \mathbb{Y} \Vdash P \Rightarrow \mathbb{Y} \Vdash Q$$

11.3.4 Negation in Kripke Semantics

$$\mathbb{X} \Vdash \neg P$$

means

$$\forall \mathbb{Y} \succeq \mathbb{X}, \quad \mathbb{Y} \nVdash P$$

11.4 Correspondence with the Topos

This Kripke semantics coincides with the internal logic of the topos described in the previous section. The correspondence is as follows:

Topos	Kripke
sieve	upward-closed set of truths
stability	monotonicity
truth	validity under all extensions

Therefore,

```
Test-centrism can be formalized as a presheaf topos,
and its internal logic can be interpreted as Kripke semantics
```

11.5 Philosophical Meaning

The significance of this structure is clear:

```
Worlds = states in the progression of tests
Knowledge = the accumulation of tests
Truth = propositions that remain valid under all future tests
```

From this perspective, the following traditionally distinct domains are unified:

- Concepts: their practical consequences in given situations
- Software: behavior in response to tests
- Scientific theories: predictions with respect to experiments

This provides a unified account of:

- concepts (practical consequences)
- programs (stability under tests)
- scientific theories (falsifiability)

12 Social Theory

A social theory of Test-centrism has not yet been fully developed. The following are preliminary ideas:

- Conventional social transformation has followed the pattern: "theory → practice (revolution, policy, movement)."
- However, this approach has too many problems:
  - The cost of failure is too high
  - The unit of change is too large
  - Verification tends to be neglected
  - Once implemented, social transformations are difficult to reverse
- Therefore, in Test-centrism, we adopt: "theory → test → practice." In other words, once a theory is formulated, tests must be written. Only when tests fail should practice be carried out.
- Practice permits only the minimal changes necessary to pass the tests.
- Refactoring must also be performed, i.e., changes that preserve behavior while minimizing complexity.

13 Political theory or the theory of tolerance

A political theory of Test-centrismas, as well as a political theory, has not yet been fully developed. The following are preliminary ideas:

13.1 Starting Point

The traditional dilemma is as follows:

- If thought itself is regulated → repression and totalitarianism
- If only actions are considered → it is often too late to prevent harm

Therefore we need a framework that resolves this tension.

13.2 Fundamental Principles

13.2.1 Principle 1: Freedom of Thought

Thought is to be tolerated without restriction

- The inner domain is inviolable
- Thoughts are not judged as true or false in a normative sense
- Epistemologically and ontologically neutral

13.2.2 Principle 2: Evaluation Based on Consequences

The object of evaluation is not the thought itself, but its consequences

This is the core of TC:

- Meaning = testable differences (pragmatism)
- Theory = a set of tests (TC)

13.3 Test Structure (Formalization)

Reorganizing your formulation:

- Input: i (situation, institution, or application of thought)
- Thought: thought(i)
- Output: o

13.3.1 Two Types of Tests

■(A) Predictive Test (open test)

$$X = (i, _)$$

- thought(i)
- Predicted outcomes

■(B) Observational Test (closed test)

$$X = (i, o)$$

- Actual outcomes

13.4 Regulatory Principle (Core Proposition)

13.4.1 TC Regulatory Principle

A thought T becomes regulatable if:

■(1) Ex Ante Regulation (Prediction-Based)

$$\text{thought}(i) \Rightarrow \text{serious social harm}$$

- Preventive in nature
- Must be applied with caution

■(2) Ex Post Regulation (Outcome-Based)

$$o \Rightarrow \text{actual social harm}$$

- Stronger justification
- Serves as the primary standard

Conclusion: OR condition

- Either condition may justify regulation
- However, their weights differ

13.5 Three-Layer Structure (Important)

TC distinguishes three levels:

13.5.1 Level 1: Thought

- Completely free
- Not subject to regulation

13.5.2 Level 2: Interpretation

- thought(i)
- Mapping from thought to action

This is the main battleground

13.5.3 Level 3: Action

- Produces actual consequences
- Strongly regulatable

13.6 Definition of Tolerance (TC Version)

13.6.1 TC Tolerance

Thoughts are fully tolerated unconditionally, while their consequences are subject to conditional intolerance.

13.6.2 Key Implication

- No exclusion of persons
- Selection occurs at the level of actions and interpretations

13.7 Response to Classical Problems

13.7.1 Response to Popper's Paradox of Tolerance

Traditional view:

If intolerance is tolerated, society collapses

TC view:

- Intolerant thoughts → tolerated
- Intolerant actions → rejected

This avoids over-repression while resolving the paradox

13.8 Core Insight (Central Thesis)

What is regulated is not thought itself, but the mapping from thought to action (interpretation)

Formally:

- What is not allowed is:

$$\text{thought} \rightarrow \text{harmful action}$$

not the thought itself.

13.9 Advantages

13.9.1 Maximum freedom

- Inner thought remains completely free

13.9.2 Precise regulation

- Targets actions, not persons

13.9.3 Avoidance of social division

- Coexistence of identical thoughts with different outcomes

13.9.4 Consistency with empiricism

- Based on testing (prediction + observation)

13.10 Weaknesses (Important Issues to Address)

13.10.1 Validity of prediction

- Who evaluates thought(i)?

13.10.2 Risk threshold

- What level of harm justifies regulation?

13.10.3 Boundary of interpretation

- The distinction between thought and interpretation may blur

13.10.4 Risk of power abuse

- Predictive regulation may be abused to suppress thought

13.11 One-Sentence Summary

> **Freedom in thought, responsibility in consequences.**

or

> **Freedom belongs to thought; responsibility belongs to consequences.**

14 Conclusion

This paper proposed **Test-centrism** as a framework for understanding objects through tests.

In Test-centrism, an object is represented by its behavior in response to inputs, and a test is defined as a finite ordered sequence of inputs, optionally including observed values when they exist. Within this setting, the following two fundamental relations concerning tests are introduced.

- **Discriminability**: the behavior of an object is defined for the input of a test
- **Verifiability**: the behavior of an object can be compared with an observed result

Based on this distinction, the fundamental concepts are formalized as follows.

- **Meaning**: behavior restricted to a test sequence
- **Discriminative equivalence**: agreement of behavior for all test inputs
- **Verification equivalence**: satisfying all observed tests
- **Understanding**: the ability to predict responses to a test sequence

Furthermore, refactoring in software development can be understood in this framework as follows.

- **Refactoring**: a transformation that preserves verification equivalence while reducing complexity

Test-centrism provides a unified framework for

- software (program behavior)
- scientific theories (empirical verification)
- philosophy (conceptual meaning)

Therefore, the notion of a test is not merely a technique in software development, but can be understood as a general methodology for understanding objects.

Categorically, Test-centrism is formulated as a presheaf topos, whose internal logic is intuitionistic and corresponds to Kripke semantics. In this setting:

```
Truth = stability under test extensions
```

From this perspective, the following traditionally distinct domains are unified:

- Concepts: their practical consequences in given situations
- Software: behavior in response to tests
- Scientific theories: predictions with respect to experiments

Furthermore, this theory integrates the following under a common formal framework:

- Peirce's pragmatism
- Popper's falsificationism

Ultimately, Test-centrism can be summarized by the following principles:

```
Meaning = response to tests
Truth = stable validity with respect to tests
Understanding = the ability to respond to tests
```

Therefore, a test is not merely a technique of software development, but a **general epistemological framework** for understanding objects, assigning meaning, and evaluating theories.